

# Log in to the Car and Viewing the Code

---

## Log in to the Car and Viewing the Code

- 0. Course Content
  - 1. How to Connect the Car to the Network
    - 1.1 Wired Connection
    - 1.2 Wireless Connection
  - 2. Connecting to the Car
    - 2.1 SSH Connection Method
    - 2.2 VNC Connection Method
      - 2.2.1 Orin Nano without a Screen
      - 2.2.2 Logging into the Car
  - 3. Viewing the Code
    - 3.1 Visual Studio Code
    - 3.2 jupyter-lab
  - 4. Source Code Workspace Explanation

## 0. Course Content

---

[!IMPORTANT]

The content of this section is very important. It covers the basic skills for editing program files and logging into the car, which are fundamental for subsequent tutorials.

- Master several ways to log in to the car and view the code.
- Recommended workflow for this chapter: Connect to the car's hotspot/insert a network cable to get an IP → Check the IP on the OLED screen → Log in via VNC → Disable the self-generated hotspot and connect to your own Wi-Fi (for running large models later) → Check the updated IP on the OLED → Reconnect via VNC → View the source code.
- **The default hotspot name for the factory image is: ROSMASTER-M3, the password is: 12345678, and the default IP is: 192.168.8.88.**

## 1. How to Connect the Car to the Network

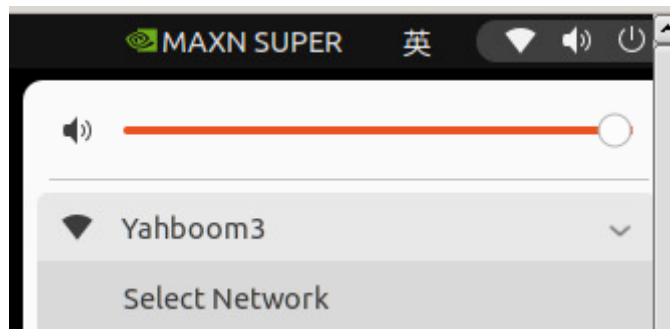
---

### 1.1 Wired Connection

- Insert a network cable into the mainboard's network port (ensure the cable has internet access for the car to connect to the internet).

### 1.2 Wireless Connection

Note: Different mainboards only have different UI icons, but the method for connecting to Wi-Fi is basically the same. Click the Wi-Fi icon, then click to select a network, and enter your Wi-Fi password (ensure the Wi-Fi has internet access for the car to connect to the internet).



## 2. Connecting to the Car

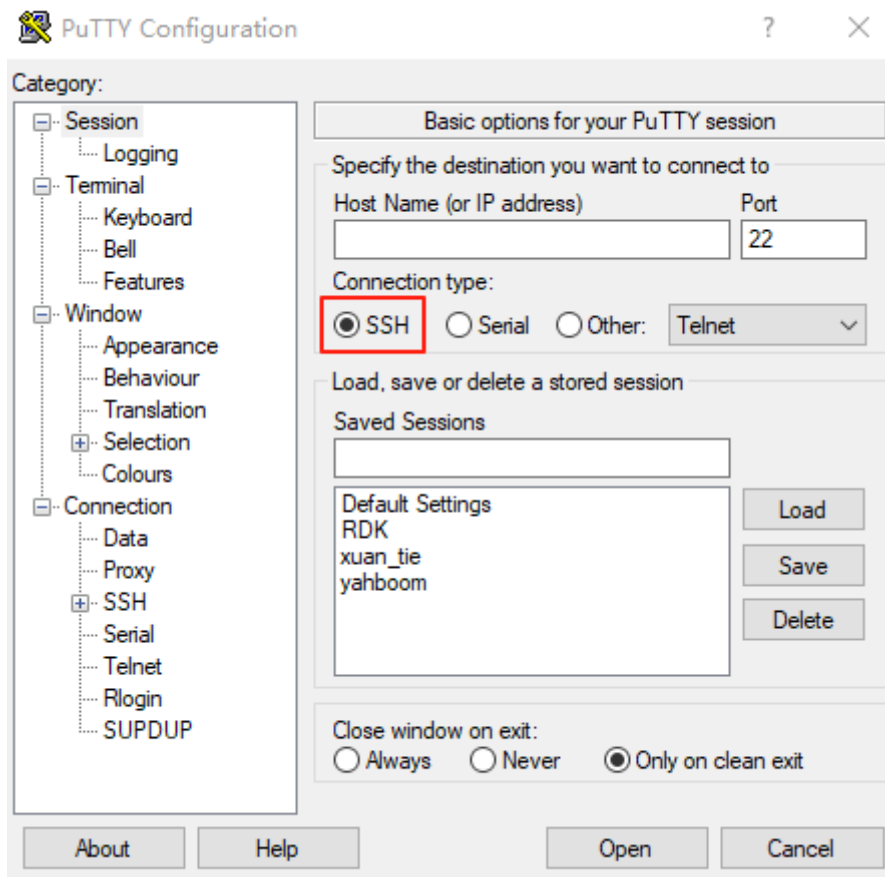
Regardless of the connection method, both the computer and the car need to be on the same local area network (LAN). The simplest way to achieve this is to connect to the same Wi-Fi or hotspot. First, connect to the car's default hotspot, ROSMASTER-M3, with the Wi-Fi password `12345678`. After a successful connection, you can use the following methods to log in.

You can also directly connect a network cable to the mainboard. The OLED screen will automatically update the IP address after connecting. The following operations are demonstrated using the IP address obtained after connecting the network cable.

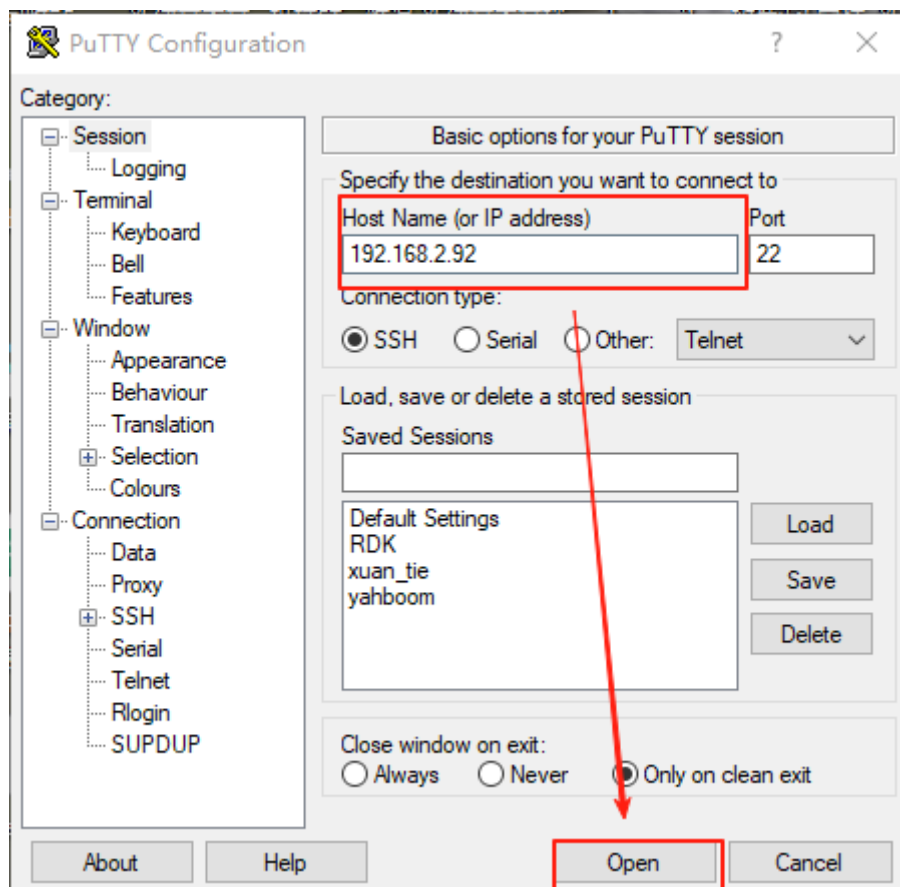
### 2.1 SSH Connection Method

[!TIP]

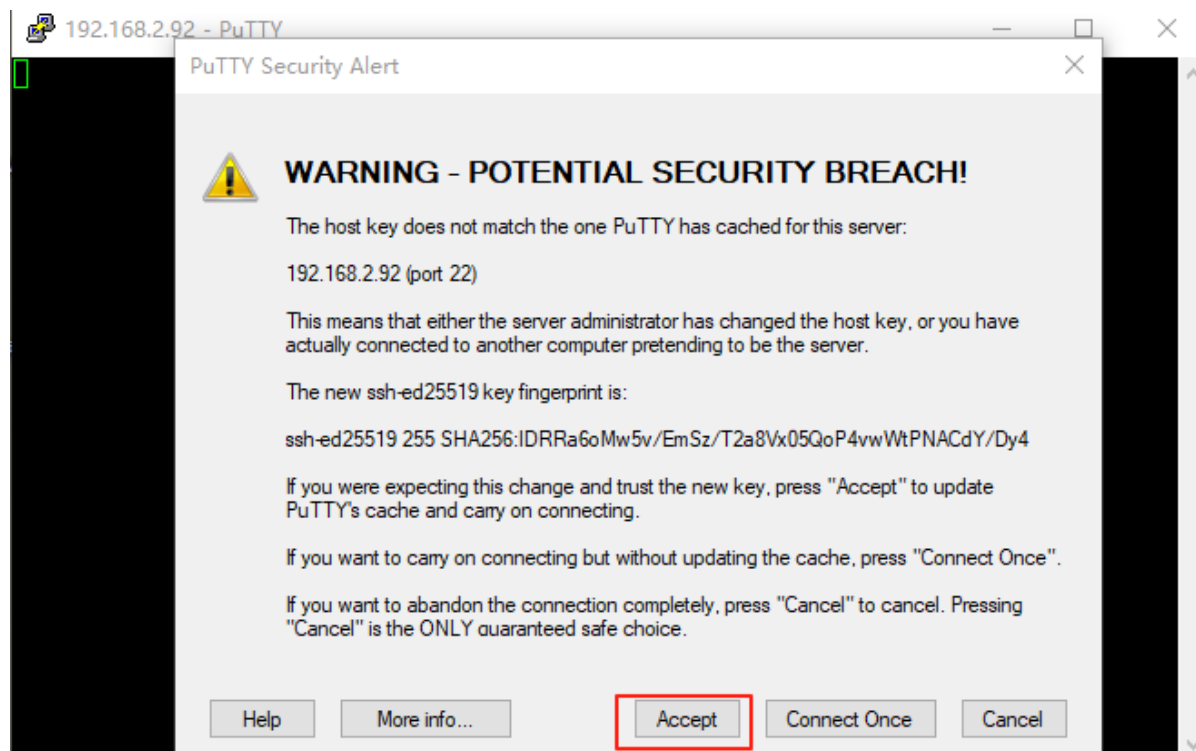
- There are many SSH connection tools available. Here, we recommend tools for Windows systems. MacOS and Linux users can choose other SSH tools.
- Use PuTTY, MobaXterm, or another SSH login tool to connect to the car. We will use PuTTY as an example. You can download PuTTY here: [Download PuTTY: latest release \(0.83\)](#)
- Choose and install the version corresponding to your computer. After successful installation, double-click to open it. The PuTTY interface is shown below:



Select **SSH**, and then enter the IP address displayed on the OLED screen in the Host Name (or IP address) field. My IP address here is 192.168.2.92, so I enter it as shown below:



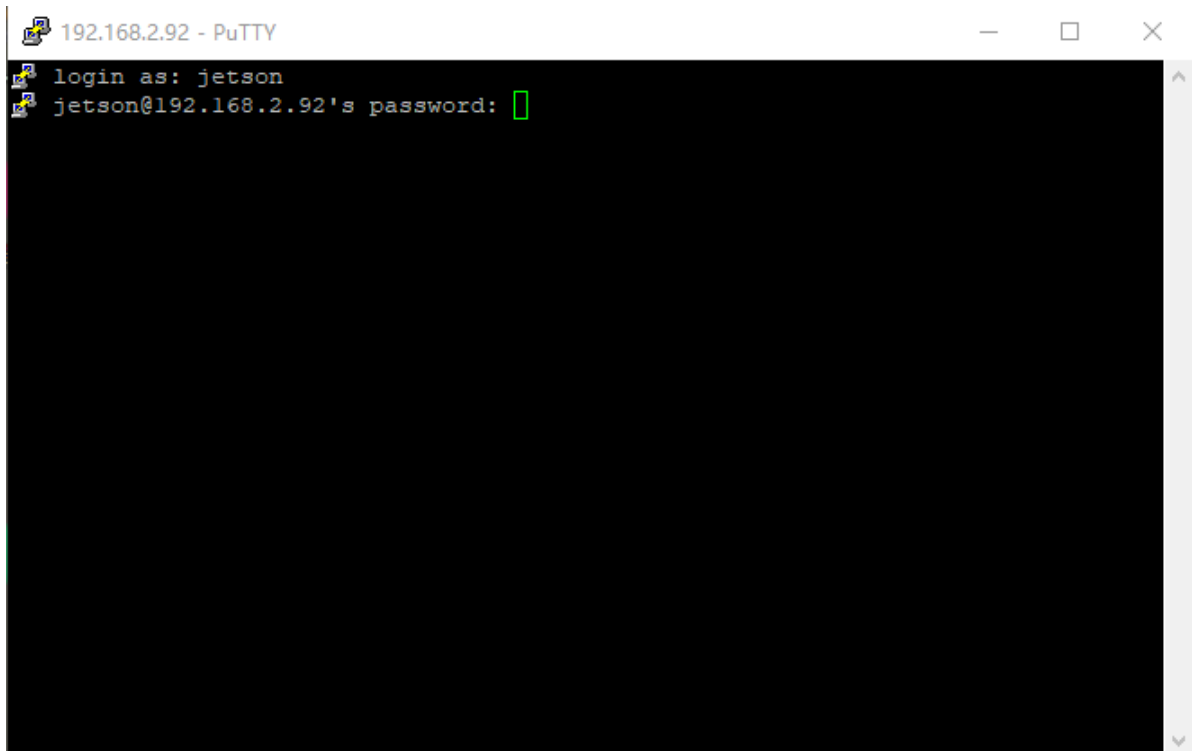
Next, click Open. A terminal interface and a pop-up window will appear. Click Accept, as shown below:



The terminal will then display **login as**. Enter the username for the car's mainboard and press Enter. You will then be prompted for a password. Enter the password. The usernames and passwords for each mainboard are as follows:

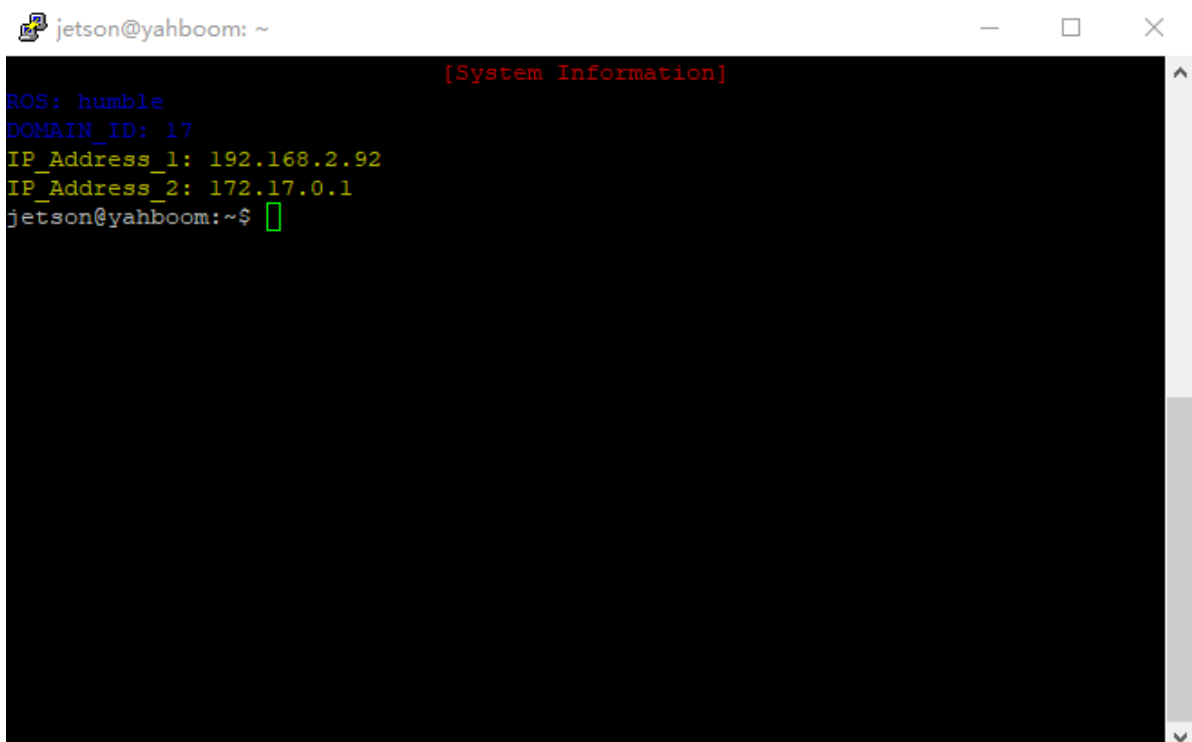
| Mainboard      | Username | Password |
|----------------|----------|----------|
| Raspberry Pi 5 | pi       | yahboom  |
| RDk X5         | sunrise  | yahboom  |
| Orin-Nano      | jetson   | yahboom  |
| Orin-NX        | jetson   | yahboom  |

Assuming my mainboard is an Orin-Nano, I would enter `jetson`, press Enter, and then enter the password `yahboom`. **Nothing will be displayed as you type the password.** Press Enter.



```
192.168.2.92 - PuTTY
login as: jetson
jetson@192.168.2.92's password: 
```

A successful connection to the car looks like this:



```
jetson@yahboom: ~
[System Information]
ROS: humble
DOMAIN_ID: 17
IP_Address_1: 192.168.2.92
IP_Address_2: 172.17.0.1
jetson@yahboom:~$ 
```

This only opens a terminal and does not display a graphical interface. Therefore, SSH is suitable for logging in when you don't need to start graphical programs and is ideal for running non-GUI examples and operating the car via commands.

## 2.2 VNC Connection Method

### 2.2.1 Orin Nano without a Screen

[!IMPORTANT]

- **Orin Nano users** without a screen should refer to this section. Other mainboard users can ignore this part.

- Since the Orin mainboard uses Ubuntu 22.04, this system requires a screen to enter the graphical interface. Users who did not purchase a screen package may have trouble opening the visual interface. Here, we provide a method for displaying the visual interface using a virtual desktop, intended only for Orin mainboard users without a connected screen. Orin users with a screen, as well as Jetson Nano and Raspberry Pi users, can skip this step.

Open the `Data-19.Attachments-Virtual Desktop Files`. Copy `xorg.conf.backup_dp` and `xorg.conf.backup_vnc` to the `/etc/X11` directory.

Install the virtual desktop environment `xserver-xorg-video-dummy`:

```
sudo apt-get install xserver-xorg-video-dummy
```

Open a terminal and enter the following command to open the virtual desktop and switch to VNC mode:

```
sudo cp /etc/X11/xorg.conf.backup_vnc /etc/X11/xorg.conf
```

Reboot the system, and you will be able to enter the desktop without a screen (continue with the VNC connection tutorial in 2.2.2).

```
sudo reboot
```

Note: After switching to VNC mode, the DP connection will be disabled (i.e., connecting a screen to the mainboard will not display anything). You need to revert the configuration to use the DP interface normally.

To disable the virtual desktop:

If you need to connect a DP display, open a terminal and enter the following command to switch to DP display mode:

```
sudo cp /etc/X11/xorg.conf.backup_dp /etc/X11/xorg.conf
```

Reboot the system, and you can now use a DP cable to connect a display.

```
sudo reboot
```

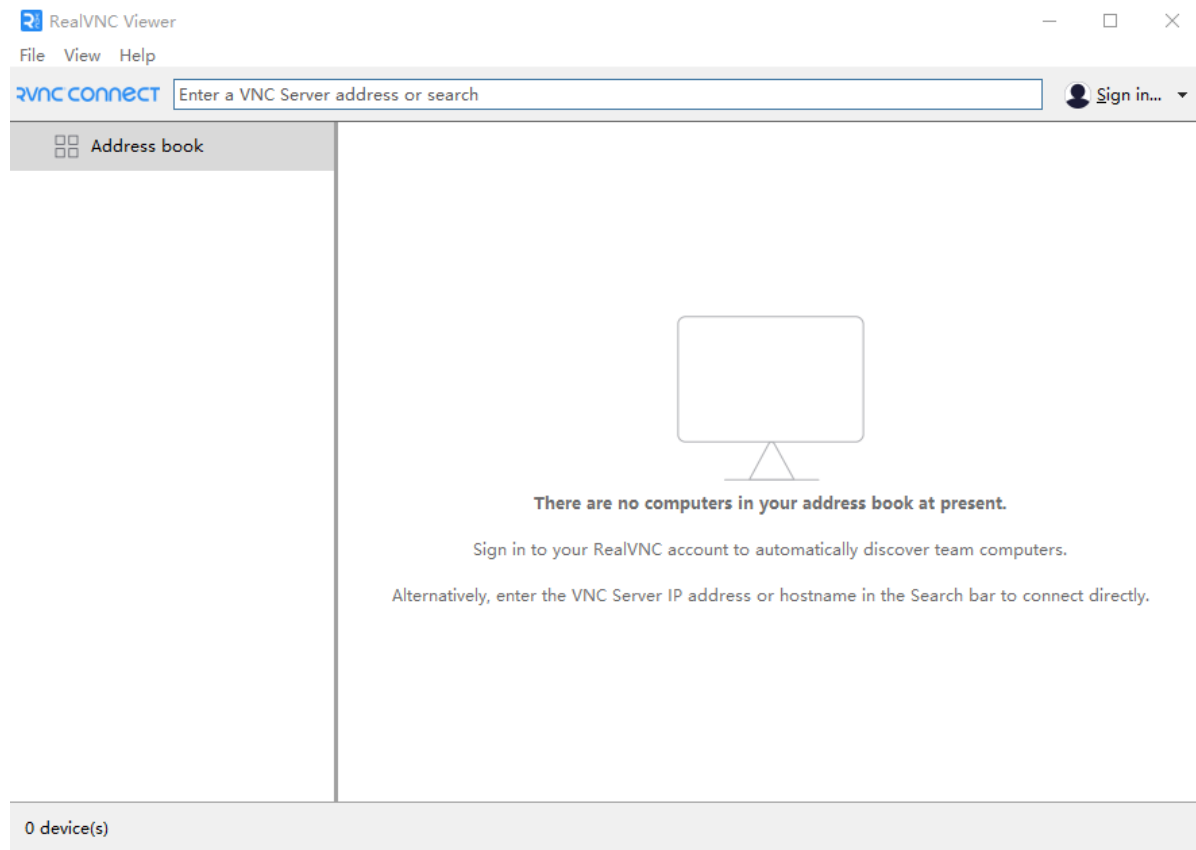
## 2.2.2 Logging into the Car

[!TIP]

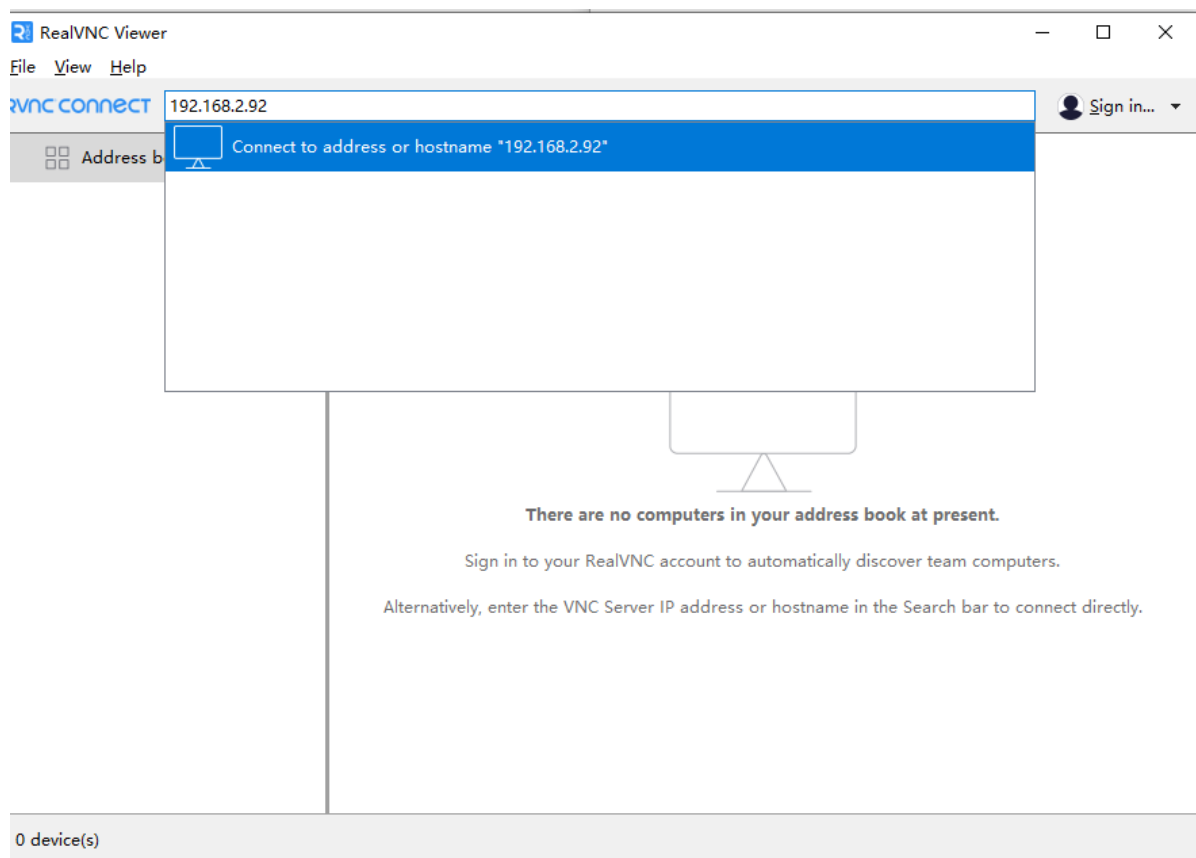
There are many VNC connection tools. Here, we recommend a tool for Windows. MacOS and Linux users can choose other VNC tools.

- VNC allows users to remotely access and control another computer's desktop environment over a network. So, when we need to access the car's desktop environment, for example, to start a graphical display, we can use VNC to connect. You can download VNC here: [Download VNC Viewer by RealVNC®](#)

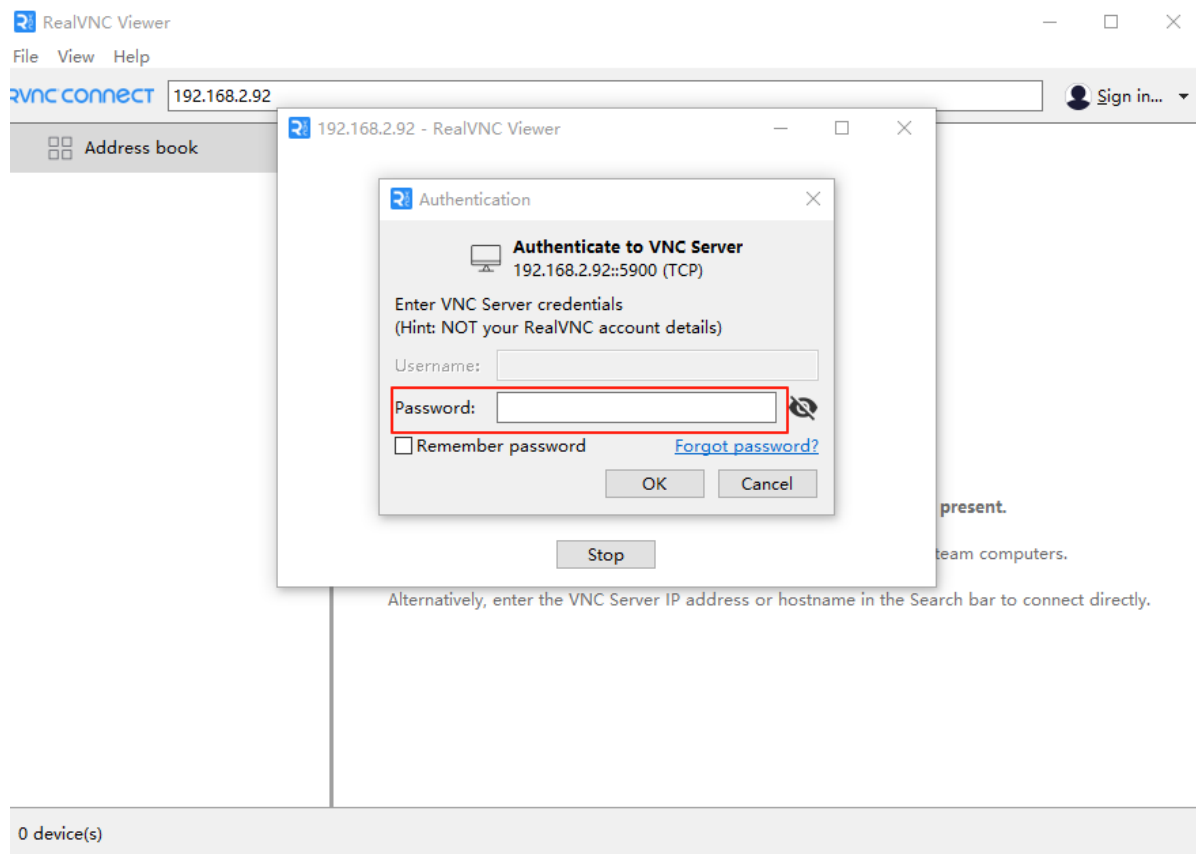
Download and install the version for your computer. After installation, double-click to open it. The screen will look like this:



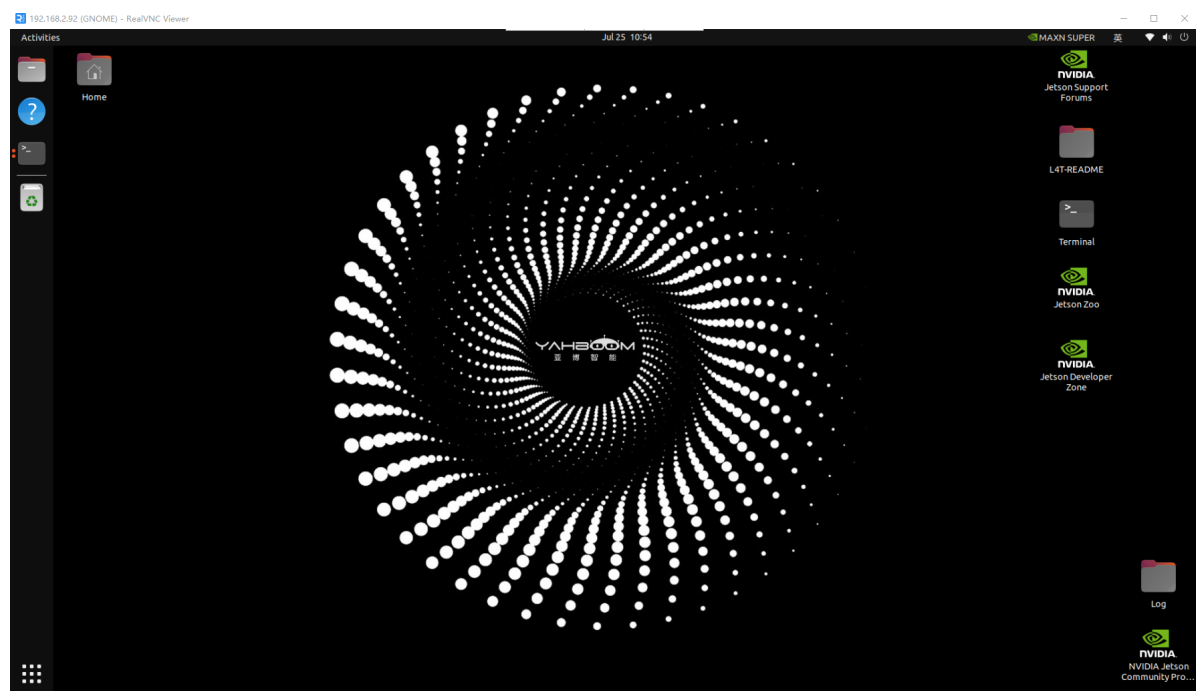
Enter the car's IP address. My IP here is 192.168.2.92, as shown below:



Then press Enter.

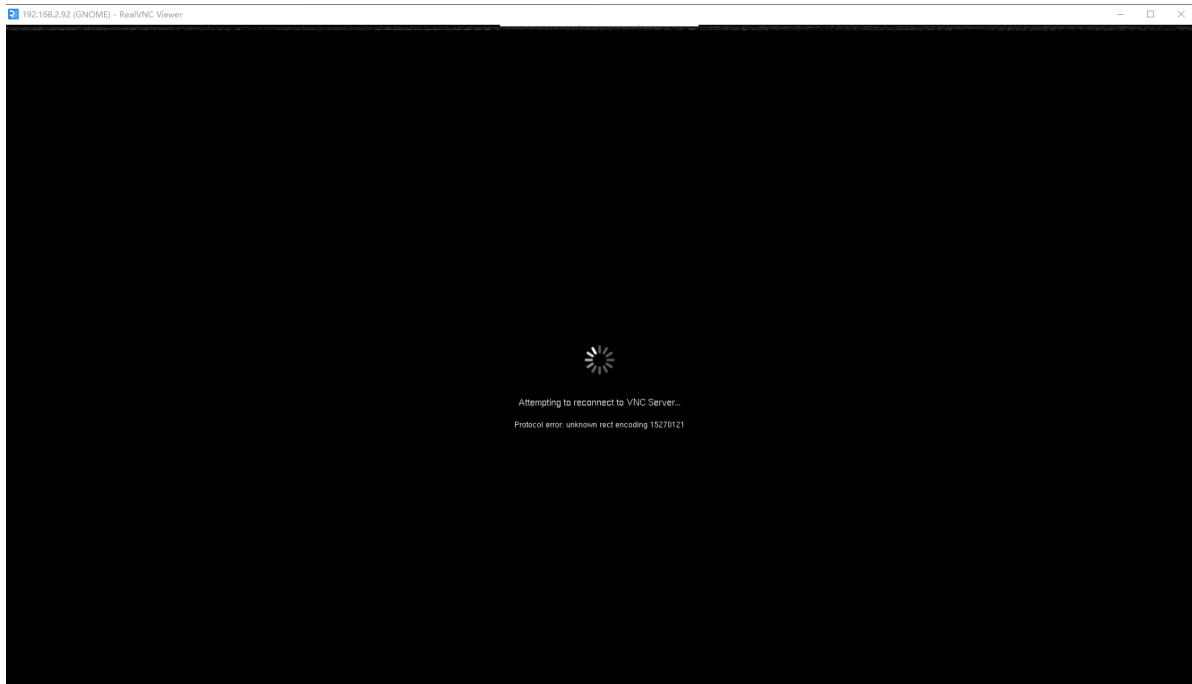


- Enter the username in the `Username` field and the password in the `Password` field (refer to the table in 2.1; the password for all mainboards is `yahboom`), then click OK to enter the desktop, as shown below:

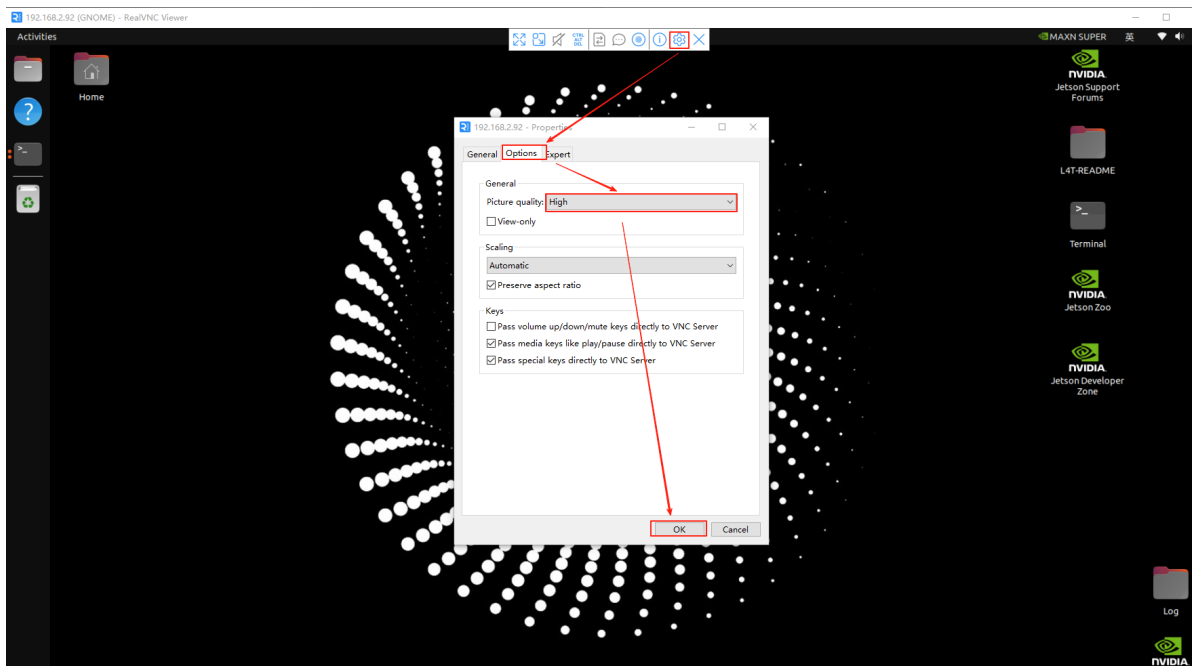


If you see an abnormal screen, such as the following:





And it keeps disconnecting, you need to adjust the settings as shown below:



Then reconnect. The Orin mainboard can only have one remote desktop connection at a time. If the connection fails, check if a remote desktop is already connected.

After connecting, since large AI models require an internet connection, and the self-generated hotspot is only a local network with no actual data, you need to switch networks to use the AI model features later.

### 3. Viewing the Code

[!IMPORTANT]

- This part is very important for users who need to develop or view the source code.
- If you only need to view and edit configuration files, the easiest way is to connect via SSH and use the `nano` tool.

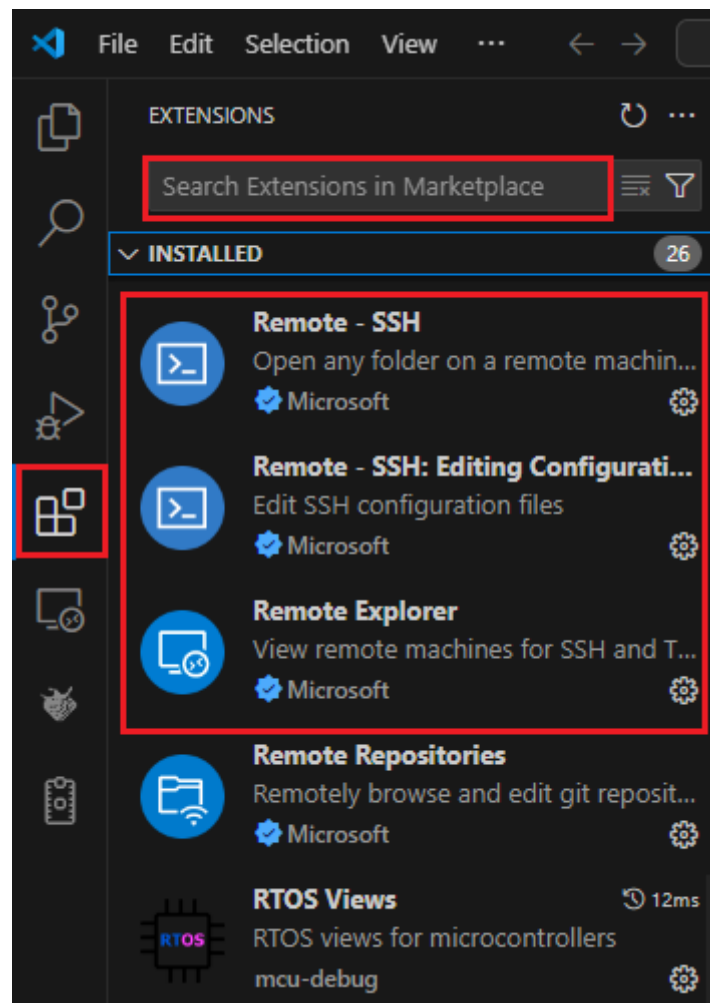
- The tools mentioned here are just recommendations. You can use any editing tool you prefer.

## 3.1 Visual Studio Code

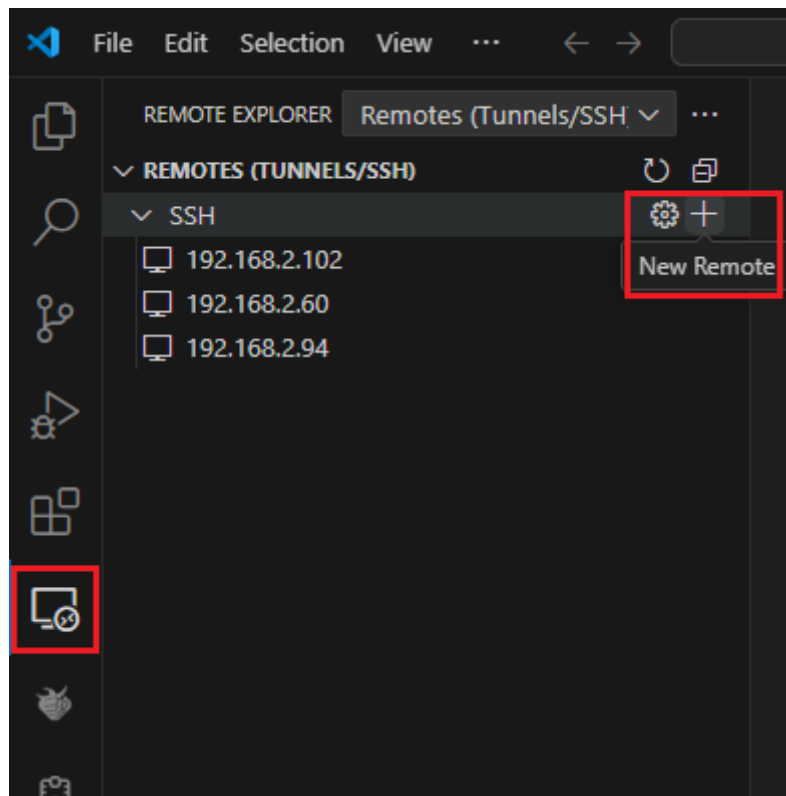
- Visual Studio Code is a lightweight, efficient, and extensible development tool with integrated core development tools, making it one of the most popular development tools in the world.

Official website: <https://code.visualstudio.com/>

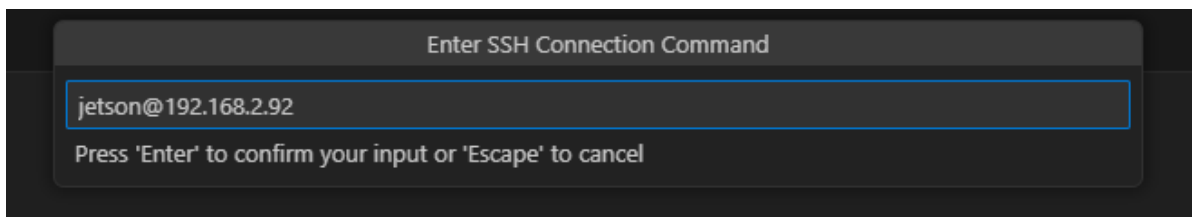
- If you don't have Visual Studio Code installed, you can download and install it from the link above.
- Start Visual Studio Code, click on the Extensions icon on the left, and install the `Remote-SSH`, `Remote - SSH: Editing Configuration Files`, and `Remote Explorer` extensions.



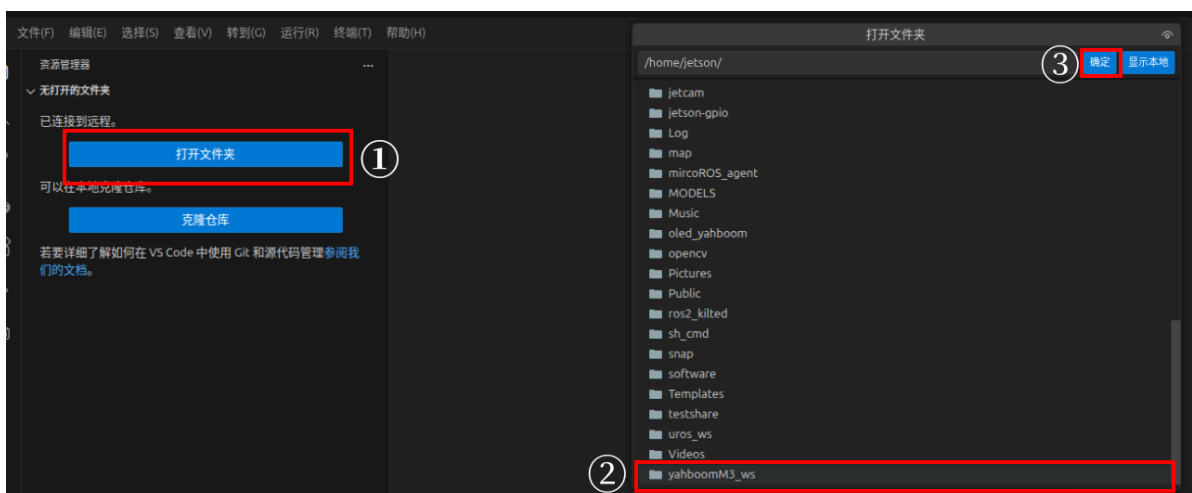
- Then click the Remote Explorer icon on the left and click to create a new remote connection.



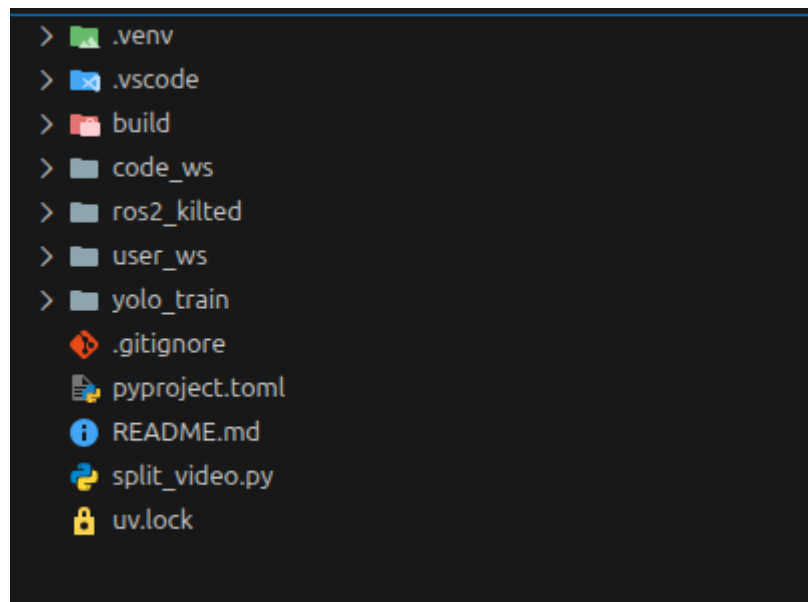
- In the pop-up input box, enter `username@IP_address` based on your mainboard, press Enter, and then enter the user password `yahboom` to log in to the car.



- Then click `open Folder`. In the pop-up window, you can select the folder you want to view. Here, we use `yahboomM3_ws` (the core source code directory for ROSMASTER-M3) as an example, and then click OK.



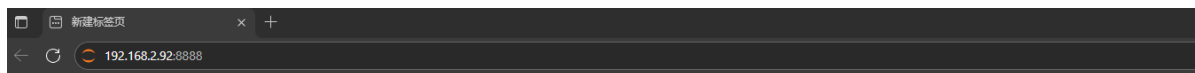
- After that, you can see all the files in the folder in the Explorer on the left.



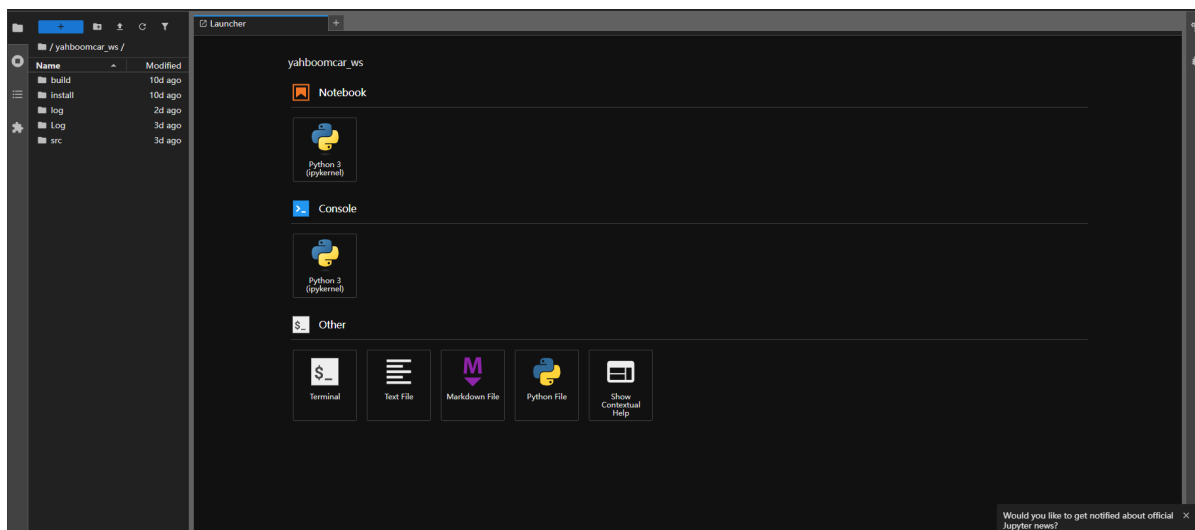
Below are several methods for viewing code, which will be convenient for editing and modifying code later.

## 3.2 jupyter-lab

Jupyter-lab starts automatically on boot, so you just need to open a browser and enter `ip_address:8888` to use Jupyter to view the code. My IP address is 192.168.2.92, so I enter the following in my browser:



Then press Enter. If prompted for a password, enter `yahboom`, and you will enter the jupyter-lab interface, as shown below:



You can view and edit the code by selecting the content in the folder on the left.

- For Raspberry Pi 5 users:

You need to enter the Docker container, then enter the command `jupyter-lab --allow-root`. In your browser, enter `car_ip:8888` to start jupyter-lab and view the car's code. As shown below, I enter the car's container, `cd` to the `/root` directory, and then enter `jupyter-lab --allow-root` in the terminal.

```
root@raspberrypi:~# cd
root@raspberrypi:~# jupyter-lab --allow-root
I 2025-07-25 11:53:53.818 ServerApp] jupyter_lab | extension was successfully linked.
I 2025-07-25 11:53:53.821 ServerApp] jupyter_server_terminals | extension was successfully linked.
I 2025-07-25 11:53:53.827 ServerApp] jupyterlab | extension was successfully linked.
I 2025-07-25 11:53:53.832 ServerApp] notebook | extension was successfully linked.
I 2025-07-25 11:53:54.148 ServerApp] notebook_shim | extension was successfully linked.
I 2025-07-25 11:53:54.154 ServerApp] notebook_shim | extension was successfully loaded.
I 2025-07-25 11:53:54.167 ServerApp] jupyter_lab | extension was successfully loaded.
I 2025-07-25 11:53:54.168 ServerApp] jupyter_server_terminals | extension was successfully loaded.
I 2025-07-25 11:53:54.170 LabApp] JupyterLab extension loaded from /usr/local/lib/python3.10/dist-packages/jupyterlab
I 2025-07-25 11:53:54.170 LabApp] JupyterLab application directory is /usr/local/share/jupyter/lab
I 2025-07-25 11:53:54.171 LabApp] Extension Manager is 'pypi'.
I 2025-07-25 11:53:54.246 ServerApp] JupyterLab | extension was successfully loaded.
I 2025-07-25 11:53:54.258 ServerApp] notebook | extension was successfully loaded.
I 2025-07-25 11:53:54.258 ServerApp] The port 8888 is already in use, trying another port.
I 2025-07-25 11:53:54.251 ServerApp] Serving notebooks from local directory: /root
I 2025-07-25 11:53:54.251 ServerApp] Jupyter Server 2.14.2 is running at:
I 2025-07-25 11:53:54.251 ServerApp] http://raspberrypi:8889/lab
I 2025-07-25 11:53:54.251 ServerApp] http://127.0.0.1:8889/lab
I 2025-07-25 11:53:54.251 ServerApp] Use Control-C to stop this server and shut down all kernels (twice to skip confirmation).
I 2025-07-25 11:53:54.251 ServerApp] Skipped non-installed server(s): bash-language-server, dockerfile-language-server-nodejs, javascript-typescript-langserver, jedi-language-server, julia-language-server, pyright, python-language-server
python-lsp-server, r-language-server, sql-language-server, texlab, typescript-language-server, unified-language-server, vscode-css-languageserver-bin, vscode-html-languageserver-bin, vscode-json-languageserver-bin, yamll-language-server
```

Then in the browser, enter the car's `ip_address:8888`. My Raspberry Pi's IP is 192.168.2.22, so I enter `192.168.2.22:8888`, as shown below:



Press Enter. If a password is required, enter `yahboom`, and you can enter jupyter-lab to view the code.

## 4. Source Code Workspace Explanation

```
├─ code_ws
├─ pyproject.toml
├─ README.md
├─ ros2_kilted
├─ split_video.py
├─ user_ws
├─ uv.lock
└─ yolo_train
```

- `code_ws`: Workspace for storing core code.
- `README.md`: File containing quick start instructions.
- `user_ws`: User code workspace for storing all startup files and configuration files.
- `ros2_kilted`: Code folder for road network planning and navigation.
- `yolo_train`: Folder for storing YOLOv11 training datasets and program files.

